

```

import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedShuffleSplit,
    StratifiedKFold, LeaveOneOut

# =====
# Load Dataset
# =====
file_path = r"C:\Users\Dell\Documents\Modified_Crime_Dataset.csv"
data = pd.read_csv(file_path)

# Recreate 'Total_Crimes' column if it doesn't exist
if 'Total_Crimes' not in data.columns:
    crime_columns = data.select_dtypes(include=['int64', 'float64']).columns
    data['Total_Crimes'] = data[crime_columns].sum(axis=1)

# Define features (X) and target variable (y)
X = data.drop(columns=['Total_Crimes', 'STATE/UT', 'DISTRICT', 'YEAR'],
    errors='ignore') # Drop unnecessary columns
y = data['Total_Crimes']

# =====
# Train-Test Split
# =====
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

print("Train-Test Split:")
print(f"Training set size: {X_train.shape}")
print(f"Test set size: {X_test.shape}\n")
# =====
# Stratified Shuffle Split
# =====
sss = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_index, test_index in sss.split(X, pd.qcut(y, 4, labels=False)):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

print("Stratified Shuffle Split:")
print("Training set size:", X_train.shape)
print("Test set size:", X_test.shape)
print("Training set class distribution:\n", pd.qcut(y_train,
    4).value_counts(), "\n")
print("Test set class distribution:\n", pd.qcut(y_test, 4).value_counts(),
    "\n")

# =====
# Stratified K-Fold

```

```
# =====
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("Stratified K-Fold Split:")
for fold, (train_index, test_index) in enumerate(kf.split(X, pd.qcut(y, 4,
    labels=False)), 1):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
    print(f"Fold {fold}:")
    print(f"Training set: {X_train.shape[0]} samples")
    print(f"Test set: {X_test.shape[0]} samples\n")

# =====
# Leave-One-Out Cross Validation (LOO)
# =====
loo = LeaveOneOut()

print("Leave-One-Out Cross Validation:")
for fold, (train_index, test_index) in enumerate(loo.split(X), 1):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
    if fold <= 5: # Display only the first 5 folds for clarity
        print(f"Fold {fold}:")
        print(f"Training set: {X_train.shape[0]} samples")
        print(f"Test set: {X_test.shape[0]} samples\n")
    if fold == 5:
        print("... Skipping remaining folds for brevity ...")
        break
```

Train-Test Split:
 Training set size: (7213, 870)
 Test set size: (1804, 870)

Stratified Shuffle Split:
 Training set size: (7213, 870)
 Test set size: (1804, 870)
 Training set class distribution:

Total_Crimes	
(-0.001, 0.389]	1804
(0.389, 0.687]	1803
(0.687, 0.973]	1803
(0.973, 16.332]	1803

Name: count, dtype: int64

Test set class distribution:

Total_Crimes	
(-0.000651, 0.39]	451
(0.39, 0.687]	451
(0.687, 0.973]	451
(0.973, 13.655]	451

Name: count, dtype: int64

Stratified K-Fold Split:

Fold 1:
Training set: 7213 samples
Test set: 1804 samples

Fold 2:
Training set: 7213 samples
Test set: 1804 samples

Fold 3:
Training set: 7214 samples
Test set: 1803 samples

Fold 4:
Training set: 7214 samples
Test set: 1803 samples

Fold 5:
Training set: 7214 samples
Test set: 1803 samples

Leave-One-Out Cross Validation:

Fold 1:
Training set: 9016 samples
Test set: 1 samples

Fold 2:
Training set: 9016 samples
Test set: 1 samples

Fold 3:
Training set: 9016 samples
Test set: 1 samples

Fold 4:
Training set: 9016 samples
Test set: 1 samples

Fold 5:
Training set: 9016 samples
Test set: 1 samples

... Skipping remaining folds for brevity ...